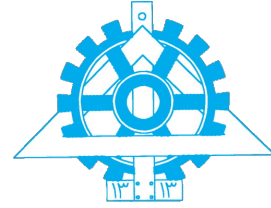




University of Tehran
College of Engineering
Department of Electrical and
Computer Engineering



Introduction to Data Science

Assignment 2

Instructors: Dr. Bahrak, Dr. Yaghoobzadeh

Prepared by (Group Submission):

Parsa Saeednia (810102460)

Mohammad Hossein Mazaheri (810102585)

Mohammad Reza Pirhadi (810102423)

May 29, 2026

Contents

1	Introduction	1
2	Q1: Database Design and Analytics (100 pts)	1
2.1	Database Design (30 pts)	2
2.2	Data Loading (10 pts)	2
2.3	Business Analytics Queries (60 pts)	2
3	Q2: Kafka Consumer Implementation (100 pts)	2
3.1	Implementation	2
3.2	Theoretical Questions	2
3.2.1	Kafka vs. Relational Databases	2
3.2.2	Stateless vs. Stateful Processing	2
3.2.3	Kafka as a Distributed Commit Log	2
4	Q3: PySpark Processing Layer (100 pts)	3
4.1	Batch Processing Layer (50 pts)	3
4.1.1	Revenue and Menu Analysis Batch Job (15 pts)	3
4.1.2	Order Time & Peak Hour Analysis (20 pts)	3
4.2	Real-Time Processing Layer (50 pts)	3
4.2.1	Spark Streaming Application	3
4.2.2	Micro-batch and Windowed Processing	3
4.2.3	Checkpoint Mechanism for Fault Tolerance	3
5	Conclusion	3

List of Figures

Introduction

This report presents the implementation of **Ashpaz**, a real-time food ordering and delivery platform developed as part of Assignment 2 for the Introduction to Data Science course. The project is structured around three interconnected layers that together form a complete data engineering pipeline.

The first layer (**Q1**) focuses on relational database design and analytics. Using the Zomato restaurant dataset, we normalize a flat CSV file into a structured schema, load the data, and answer business-critical queries covering delivery statistics, pricing tiers, and neighborhood performance.

The second layer (**Q2**) introduces **Apache Kafka** as a distributed messaging backbone. A Kafka consumer reads from the `ashpaz.order` topic, performs stateless validation on each incoming order event, and routes messages to either `ashpaz.valid` or `ashpaz.error_log` depending on data quality. This section also addresses key theoretical questions on Kafka's architecture, stateless vs. stateful processing, and the distributed commit log model.

The third layer (**Q3**) implements a **PySpark** processing layer with both batch and real-time components. The batch jobs analyze historical order data for revenue insights and peak-hour patterns, while the streaming application connects to the Kafka consumer and performs windowed micro-batch aggregations with checkpoint-based fault tolerance.

The data generator (`Ashpaz.py`) simulates realistic order traffic — including intentional noise such as invalid phone numbers, price anomalies, and spam user behavior — providing a robust test environment for the entire pipeline.

Q1: Database Design and Analytics (100 pts)

2.1 Database Design (30 pts)

[Placeholder — Database Design]

2.2 Data Loading (10 pts)

[Placeholder — Data Loading]

2.3 Business Analytics Queries (60 pts)

[Placeholder — Business Analytics Queries]

Q2: Kafka Consumer Implementation (100 pts)

3.1 Implementation

[Placeholder — Kafka Consumer Implementation]

3.2 Theoretical Questions

3.2.1 Kafka vs. Relational Databases

[Placeholder — Kafka vs. Relational Databases]

3.2.2 Stateless vs. Stateful Processing

[Placeholder — Stateless vs. Stateful Processing]

3.2.3 Kafka as a Distributed Commit Log

[Placeholder — Distributed Commit Log]

Q3: PySpark Processing Layer (100 pts)

4.1 Batch Processing Layer (50 pts)

4.1.1 Revenue and Menu Analysis Batch Job (15 pts)

[Placeholder — Revenue and Menu Analysis Batch Job]

4.1.2 Order Time & Peak Hour Analysis (20 pts)

[Placeholder — Order Time & Peak Hour Analysis]

4.2 Real-Time Processing Layer (50 pts)

4.2.1 Spark Streaming Application

[Placeholder — Spark Streaming Application]

4.2.2 Micro-batch and Windowed Processing

[Placeholder — Micro-batch and Windowed Processing]

4.2.3 Checkpoint Mechanism for Fault Tolerance

[Placeholder — Checkpoint Mechanism]

Conclusion

[Placeholder — Conclusion]